

Documentación: Arquitectura y Patrones de Diseño

Space Invaders — Proyecto de Programación

Fecha: 11 de mayo de 2026 — Grupo: Cachopin

Índice

1. Introducción General	2
Parte I: Arquitectura MVC + Observer	3
2. Introducción al Patrón MVC + Observer	3
3. Implementación en Space Invaders	3
3.1. La Capa Modelo	3
3.2. La Capa Vista	6
3.3. La Capa Controlador	7
3.4. Diagrama UML	8
3.5. Patrón Observer: Notificación y Actualización	8
3.6. Método <code>notify</code>	8
3.7. Método <code>update</code>	8
3.8. Sistema de notificaciones en Space Invaders	8
3.9. Transición de pantallas	10
Parte II: Patrones de Diseño	12
4. Introducción a los Patrones de Diseño	12
5. Factory: La Fábrica de Naves	12
5.1. La Idea	12
5.2. Evolución del patrón	12
5.3. Ventajas del patrón Factory	13
6. Strategy: Disparos Intercambiables	13
6.1. La Idea	13
6.2. Evolución del patrón	13
6.3. Ventajas del patrón Strategy	14
7. Composite: Píxeles, Naves y Disparos	15
7.1. La Idea	15
7.2. Evolución del patrón	15
7.3. Ventajas del patrón Composite	16
8. Cómo Todo Funciona Junto	17
9. Conclusiones Finales	17

Introducción General

En este proyecto se ha implementado una arquitectura basada en dos enfoques complementarios: el patrón arquitectónico **MVC (Modelo-Vista-Controlador)** combinado con el patrón **Observer** para la sincronización entre componentes, y la aplicación de tres patrones de diseño (**Factory**, **Strategy** y **Composite**) para resolver problemas específicos de flexibilidad y mantenibilidad.

El proyecto ha pasado por una evolución clara entre el **Sprint 1**, donde se implementó una versión funcional pero básica, el **Sprint 2**, donde se refactorizó profundamente el modelo para soportar múltiples naves, disparos compuestos, flota de enemigos y detección de colisiones a nivel de píxel, y la **versión final**, la versión actual y definitiva del juego. Esta documentación refleja todas las etapas y el razonamiento detrás de cada cambio.

Parte I: Arquitectura MVC + Observer

Introducción al Patrón MVC + Observer

El patrón **Modelo-Vista-Controlador (MVC)** es una arquitectura de diseño utilizada en programación orientada a objetos que permite organizar una aplicación separando sus componentes en tres partes principales. Esta separación facilita el mantenimiento del código, mejora su organización y permite que cada componente tenga una responsabilidad clara dentro del sistema.

Modelo

Se encarga de gestionar los datos y la lógica del programa. Contiene toda la lógica de negocio del juego y las operaciones necesarias para modificarla o consultarla. El modelo no depende de la interfaz gráfica ni de cómo se muestran los datos al usuario.

Vista

Es la parte encargada de presentar la información al usuario. Su función principal es mostrar los datos que proporciona el modelo y actualizarse cuando estos cambian. La vista no contiene la lógica principal del programa, sino únicamente la representación visual de la información.

Controlador

Actúa como intermediario entre el usuario y el sistema. Recibe las acciones del usuario (clics, entradas de teclado, etc.), interpreta estas acciones y solicita al modelo que realice las operaciones correspondientes.

A esta arquitectura se le añade el patrón **Observer (Observador)** para que la vista se mantenga actualizada cuando cambian los datos del modelo. Este patrón establece una relación entre un objeto denominado *observable* (el modelo) y uno o varios *observadores* (las vistas). Cuando el estado del observable cambia, todos los observadores registrados son notificados automáticamente y pueden actualizar su información.

La combinación de MVC y Observer permite desarrollar aplicaciones más modulares, donde la lógica del programa, la presentación de los datos y la interacción con el usuario están claramente separadas. Esto facilita la reutilización del código, la ampliación del sistema y el mantenimiento a largo plazo.

Implementación en Space Invaders

La Capa Modelo

La capa modelo es la que mayor evolución ha sufrido entre sprints. A continuación se describe cómo era en cada etapa y qué motivó cada cambio.

Sprint 1 — Versión inicial

La clase central era **Espacio** (Singleton y Observable). Dentro de ella vivía toda la lógica: el **game loop** con un **Timer** de 50ms, la detección de colisiones, y las condiciones de victoria y derrota.

Las clases auxiliares eran simples y directas:

- **Naves** — clase abstracta con `x`, `y`, `velocidad` y `vivo`. Definía el método abstracto `mover(int dx, int dy)`.
- **Jugador** — extendía **Naves**. Tenía un único objeto **Disparo** asociado. Al disparar, creaba un nuevo **Disparo** en la posición (`x`, `y-1`). Solo un disparo activo a la vez.
- **Enemigo** — extendía **Naves**. Implementaba `mover()` sumando directamente el desplazamiento. Un único enemigo inicial en posición aleatoria de la fila superior.
- **Disparo** — clase simple con `x`, `y` y `activo`. El método `subir()` decrementaba `Y` un píxel por tick. Se desactivaba al salir del tablero.

La detección de colisiones en **Espacio** comparaba la posición del disparo con la del enemigo con una ventana de 2 píxeles para compensar la discretización:

```
if (e.isVivo() && d.getX() == e.getX()
    && d.getY() <= e.getY() && d.getY() >= e.getY() - 1) {
    e.setVivo(false);
    d.setActivo(false);
    ...
}
```

Todo era sencillo y funcionaba, pero el modelo estaba muy acoplado: **Espacio** sabía demasiado sobre **Jugador**, **Enemigo** y **Disparo**, y cualquier extensión requería modificar directamente **Espacio**.

¿Qué limitaciones motivaron el cambio al **Sprint 2**? Al finalizar el primer Sprint y después de la reunión con el profesor, buscamos arreglar los fallos comentados, como que solo apareciese un enemigo. Además, al querer añadir múltiples tipos de nave, disparos con formas distintas, varios enemigos simultáneos con movimiento independiente y colisiones precisas píxel a píxel, el modelo del Sprint 1 no escalaba. Cada nueva funcionalidad obligaba a tocar **Espacio**, rompiendo el principio de responsabilidad única.

Sprint 2 — Actualización

En este nuevo Sprint se produjo un cambio profundo en la arquitectura. El controlador transmitiría los cambios del juego a una nueva clase **JugadorBueno**, y se introdujeron nuevos patrones que modificaron el diseño de las clases:

- **Espacio** (Singleton + Observable) — Evolucionó su papel en el proyecto: pasó de ser la clase que gestionaba toda la lógica a ser la que notificaría a la vista de los cambios lógicos del juego. En este Sprint se mantuvieron algunos aspectos clave como el Timer de 50ms y las condiciones de victoria/derrota, y seguía siendo la encargada de inicializar la nave y los enemigos mediante llamadas directas a **JugadorBueno** y **FlotaEnemigos**.
- **JugadorBueno** (Singleton + Observer) — Nueva clase encargada de recibir la información del controlador y enviar señales a las diferentes clases para cambiar

la lógica del juego. Se responsabilizaba de la construcción de la nave inicial y los cambios de disparo, aunque todavía no enviaba señales de inicialización propias.

- **FlotaEnemigos** (Singleton) — Nueva clase encargada de almacenar a todos los enemigos del juego, gestionando su movimiento y las colisiones con los disparos del jugador.
- **Naves** — Sigue siendo abstracta, pero ahora gestiona un **Composite** (árbol de píxeles que forma la nave) y un **Disparo** (gestor de estrategias). Define el método abstracto **construir()** con el template method para que cada subclase dibuje su propia geometría.
- **Nave1**, **Nave2**, **Nave3** — Subclases concretas de **Naves**. Cada una define su geometría de píxeles en **construir()** y sus estrategias de disparo disponibles.
- **Enemigo** — Ahora es multipíxel (también usa el **Composite**).

¿Qué limitaciones motivaron el cambio al Sprint 2? Al finalizar el segundo Sprint, encontramos el siguiente problema: Espacio no podía comunicarse directamente con otras clases. Sin embargo, nos interesaba que se comunicase con **FlotaEnemigos** y **JugadorBueno**, por lo que decidimos implementar el patron Observer y, de esa forma, puedan comunicarse. Además, esto significaba que la inicialización se haría mediante notificaciones, y que desde **Espacio** necesitaríamos conocer en todo momento las posiciones de todas las naves y disparos, de ahí surgió la idea de crear una matriz que guardase dichas posiciones.

Versión Final

La versión final redistribuye las responsabilidades de forma mucho más granular:

- **Espacio** (Singleton + Observable) — Sigue siendo el núcleo observable, pero ya no contiene el game loop ni la lógica de movimiento. Su función principal es mantener una **matriz espejo** (`int[100][60]`) que registra qué celda está ocupada y por qué entidad (id de enemigo, id de disparo, id de jugador). Es el árbitro de colisiones: cuando un disparo se mueve, **Espacio** consulta la matriz y, si hay colisión, notifica a los observadores. También gestiona la puntuación y las condiciones de victoria/derrota.
- **JugadorBueno** (Singleton + Observer) — Delega la creación de la nave a **NaveFactory** y actúa como intermediario entre los eventos de teclado y la nave. Observa a **Espacio** para recibir la señal de eliminación de un proyectil (mensaje tipo 19).
- **Nave4** — Subclase concreta de **Naves**. Crea una nave distinta de las 3 anteriores, pero el funcionamiento y construcción es idéntico.
- **Enemigo** — Tiene un id único para identificarlo en la matriz y en la vista.
- **FlotaEnemigos** (Singleton + Observer) — Gestiona la lista de enemigos activos. Observa a **Espacio** para recibir la señal de eliminación de un enemigo (mensaje

tipo 18) y la de inicialización de la flota (mensaje tipo 20).

- **TimerDisparo** (Singleton) — Tick cada 50ms, llama a `JugadorBueno.actualizarDisparos()`.
- **TimerEnemigo** (Singleton) — Tick cada 200ms, llama a `FlotaEnemigos.moverEnemigos()`.

Con esta distribución, **Espacio** ya no necesita conocer los detalles de cómo se mueven las naves ni cómo se crean los disparos. Cada clase tiene una responsabilidad clara y acotada.

La Capa Vista

Sprint 1 — Versión inicial

Solo existían dos vistas: **StartFrame** (pantalla de inicio con fondo negro, título en verde y mensaje «Pulsa SPACE para jugar») y **MainFrame** (cuadrícula 100x60 con fondo negro). Cada celda era un **JLabel** con color de fondo: magenta para el jugador, rojo para el enemigo, amarillo para el disparo. El fin de partida se mostraba con un **JLabel** superpuesto en la parte inferior del propio **MainFrame**, sin cambiar de pantalla.

¿Qué motivó el cambio? La pantalla de fin de partida integrada en **MainFrame** era poco elegante y no permitía mostrar la puntuación ni ofrecer al jugador la opción de volver al menú de forma limpia. Por ello, se decidió añadir un frame nuevo al acabar la partida en la versión final. Cabe destacar que esta idea solo se implementó en el Sprint 3, donde se nos ocurrió la idea de la puntuación.

Versión Final

Se añade una tercera vista, **FinalFrame**, y se enriquecen las existentes:

- **StartFrame** — Añade selección de nave (teclas 1/2/3/4), imagen de fondo con **JLayeredPane** e instrucciones detalladas. La nave elegida se almacena localmente como `tipoNavePendiente` y solo se confirma al pulsar SPACE, momento en que se llama a `JugadorBueno.getJugadorBueno().inicializar(tipoNavePendiente)`.
- **MainFrame** — El fondo pasa a ser una imagen escalada con **JLayeredPane**. Los colores de las celdas ya no son únicos: cada tipo de nave tiene su propio color (verde para Nave1, azul para Nave2, morado para Nave3, amarillo para Nave4). Se añade un **JLabel** de puntuación actualizado por el mensaje tipo 22. Al recibir los mensajes tipo 7 (Game Over) u 8 (Victoria), se desregistra como observer, se oculta y crea un **FinalFrame**.
- **FinalFrame** — Nueva pantalla de fin con imagen de fondo, mensaje «VICTORIA» o «GAME OVER» en color diferente, puntuación obtenida y opción de volver al menú pulsando SPACE.

Nota importante: el hecho de que **MainFrame** distinga entre más de veinte tipos de mensaje no implica que contenga lógica de negocio. La vista simplemente recibe

un array de enteros y, según el tipo, pinta o borra celdas. No toma ninguna decisión sobre el estado del juego.

La Capa Controlador

En todas las versiones los controladores son clases internas privadas llamadas **Controller** dentro de cada frame, implementando **KeyListener**. Esto es correcto desde el punto de vista del patrón MVC: el controlador pertenece a la vista pero no es la vista. El flujo común es: el frame crea una instancia del controlador, la registra con `addKeyListener(controller)` y solicita el foco con `requestFocusInWindow()`. Solo `keyPressed` contiene lógica real; `keyReleased` y `keyTyped` quedan vacíos.

Sprint 1 — Versión inicial

- **Controller de StartFrame:** solo detectaba SPACE y llamaba a `Espacio.getEspacio().cambiarAMain()`. Controller de MainFrame: traducía las cuatro flechas en llamadas a `Espacio.getEspacio().moverJugador(dx,dy)` y SPACE en `Espacio.getEspacio().disparar()`.

El controlador hablaba directamente con `Espacio`, que era el único punto de entrada al modelo.

¿Qué motivó el cambio? Al introducir `JugadorBueno` como singleton que encapsula la nave y sus acciones, `Espacio` dejó de ser el punto de entrada para las acciones del jugador. El controlador debía actualizarse para reflejar esta nueva distribución de responsabilidades. El controlador solo fue cambiado en el segundo Sprint, debido al cambio de rol de la clase `Espacio` y la creación de la nueva clase `JugadorBueno`. De tal forma, la versión del segundo Sprint, en cuanto al Controller respecta, es la versión final de nuestro proyecto.

Versión Final

- **Controller de StartFrame:** detecta las teclas 1/2/3/4 para actualizar `tipoNavePendiente` (solo en la vista, sin tocar el modelo) y SPACE para llamar a `JugadorBueno.getJugadorBueno().inicializar(tipoNavePendiente)`. El modelo ya no necesita saber de la transición de pantalla.
- **Controller de MainFrame:** las flechas llaman a `JugadorBueno.getJugadorBueno().mover(dx,dy)`, SPACE llama a `JugadorBueno.getJugadorBueno().disparar()`, y la tecla M llama a `JugadorBueno.getJugadorBueno().cambiarTipoDisparo()`. El controlador ya no llama a `Espacio` directamente para las acciones de juego.
- **Controller de FinalFrame:** detecta SPACE y llama al método privado `volverAlMenu()`, que desregistra el observer, oculta la ventana y crea un nuevo `StartFrame`.

Importante: en ninguna versión el controlador verifica si la partida ha terminado. Esa validación ocurre en el modelo (`Espacio`), que es quien tiene autoridad sobre el estado del juego.

Diagrama UML

Patrón Observer: Notificación y Actualización

Método notify

El método `notify` pertenece al **observable** (el modelo). Su función es avisar a todos los observadores registrados de que el estado ha cambiado. El observable no necesita conocer los detalles internos de sus observadores: simplemente los recorre y llama a su `update`, manteniendo un bajo acoplamiento entre las clases.

En Java, se implementa siempre con la secuencia obligatoria:

```
setChanged(); // Marca el observable
              como modificado
notifyObservers(new int[]{tipo, ...}); // Notifica y pasa datos
```

Sin `setChanged()`, `notifyObservers()` no propaga nada.

Método update

El método `update` pertenece a los **observadores** (las vistas y ciertos componentes del modelo). Cada observador define aquí cómo reacciona al cambio. Recibe el objeto observable y el argumento pasado por `notifyObservers`.

En este proyecto, el argumento es siempre un `int[]`: el primer elemento identifica el tipo de notificación y los siguientes contienen los datos necesarios (coordenadas, ids, puntuación).

Sistema de notificaciones en Space Invaders

Sprint 1 — Versión inicial

El sistema de mensajes era simple: 10 tipos (0–9), todos gestionados directamente en Espacio. Cada tipo correspondía a una acción concreta y visible en la cuadrícula:

- **Tipo 0** — Movimiento del jugador: {0, oldX, oldY, newX, newY}
- **Tipo 1** — Nuevo disparo: {1, newX, newY}
- **Tipo 2** — Movimiento del disparo: {2, oldX, oldY, newX, newY}
- **Tipo 3** — Disparo sale del tablero: {3, oldX, oldY}
- **Tipo 4** — Movimiento de enemigo: {4, oldX, oldY, newX, newY}
- **Tipo 5** — Colisión disparo-enemigo: {5, disparoX, disparoY, enemigoX, enemigoY}
- **Tipo 6** — Inicialización del juego: {6, jugadorX, jugadorY, enemigoX, enemigoY}
- **Tipo 7** — Game Over: {7}
- **Tipo 8** — Victoria: {8}
- **Tipo 9** — Cambio a MainFrame: {9}

Cada notificación correspondía a una única celda (o par de celdas), lo que era suficiente para representar un jugador de un píxel y un enemigo de un píxel.

¿Qué motivó el cambio? Al pasar a naves multipíxel, un solo par de coordenadas ya no era suficiente. Además, los disparos compuestos (flecha, rombo) ocupan varias celdas simultáneamente. Fue necesario ampliar el protocolo de mensajes y separar la notificación de la vista de la notificación interna de gestión.

Versión Final

El sistema de mensajes se amplía a más de veinte tipos. Una distinción clave es entre mensajes **de vista** (tipos 1–3, 10, 12, 14–17, 21, 22), que `MainFrame` procesa para pintar o borrar celdas, y mensajes **de modelo** (tipos 18, 19, 20), que `FlotaEnemigos` y `JugadorBueno` procesan para actualizar sus listas internas. Los tipos 7, 8 y 9 son recibidos tanto por la vista como por el modelo.

- **Tipo 1** — Nuevo píxel de disparo: {1, x, y}
- **Tipo 2** — Movimiento de píxel de disparo: {2, oldX, oldY, newX, newY}
- **Tipo 3** — Píxel de disparo sale del tablero: {3, oldX, oldY}
- **Tipo 7** — Game Over: {7, puntuacion}
- **Tipo 8** — Victoria: {8, puntuacion}
- **Tipo 9** — Cambio a MainFrame: {9}
- **Tipo 10** — Borrar celda de nave jugadora: {10, x, y}
- **Tipo 12** — Borrar píxel de enemigo: {12, x, y}
- **Tipo 14** — Pintar píxel de enemigo (color genérico): {14, x, y}
- **Tipo 15** — Pintar píxel de Nave1 (verde): {15, x, y}
- **Tipo 16** — Pintar píxel de Nave2 (azul): {16, x, y}
- **Tipo 17** — Pintar píxel de Nave3 (morado): {17, x, y}
- **Tipo 18** — Eliminar enemigo de la flota (solo modelo): {18, enemigoId}
- **Tipo 19** — Eliminar disparo (solo modelo): {19, disparoId}
- **Tipo 20** — Inicializar flota de enemigos: {20, n_enemigos}
- **Tipo 21** — Pintar píxel de Nave4 (amarillo): {21, x, y}
- **Tipo 22** — Puntuación actualizada: {22, puntuacion}

Nótese que en la versión final ya no existe el tipo 4 (movimiento de enemigo como celda única) ni el tipo 6 (inicialización como dos celdas): ahora el movimiento y la inicialización de enemigos multipíxel se notifican píxel a píxel mediante los tipos 12 y 14.

Transición de pantallas

Sprint 1 — Versión inicial

El flujo era lineal y sencillo:

1. Usuario pulsa SPACE en **StartFrame**.
2. Controller llama a `Espacio.getEspacio().cambiarAMain()`.
3. `cambiarAMain()` inicializa jugador y enemigo, arranca el timer, notifica tipo 9 y luego tipo 6.
4. `StartFrame.update()` recibe tipo 9: se desregistra, se oculta y crea **MainFrame**.
5. **MainFrame** recibe tipo 6 y pinta las posiciones iniciales.
6. Al acabar la partida (tipo 7 u 8), **MainFrame** muestra un **JLabel** con el resultado. No hay nueva pantalla.

¿Qué motivó el cambio? Mostrar el resultado como texto superpuesto era una solución provisional que no permitía mostrar la puntuación final ni ofrecer al usuario la opción de volver al menú de forma limpia.

Versión Final

El flujo completo de transiciones queda así:

StartFrame → MainFrame:

1. Usuario elige nave (teclas 1/2/3/4); solo se actualiza `tipoNavePendiente` en la vista.
2. Usuario pulsa SPACE: Controller llama a `JugadorBueno.getJugadorBueno().inicializar(tipoNavePendiente)`.
3. `JugadorBueno.inicializar()` crea la nave vía `NaveFactory`, llama a `nave.inicializar()` que propaga hasta `Espacio.cambiarAMain()`.
4. `Espacio.cambiarAMain()` notifica tipo 9.
5. `StartFrame.update()` recibe tipo 9: se desregistra, se oculta y crea **MainFrame**.
6. `JugadorBueno` registra la posición inicial en la matriz y la vista (notificaciones tipo 15/16/17/21).
7. `Espacio` notifica tipo 20 para que `FlotaEnemigos` inicialice la flota.

MainFrame → FinalFrame:

1. `Espacio` detecta game over o victoria y notifica tipo 7 u 8 con la puntuación.
2. `MainFrame.update()` recibe el tipo: se desregistra como observer, se oculta y crea `new FinalFrame(victoria, puntuacion)`.

FinalFrame → StartFrame:

1. Usuario pulsa SPACE.
2. Controller llama a `volverAlMenu()`: desregistra observer, oculta y crea StartFrame.

En ninguna de estas transiciones una vista conoce directamente a otra. Todas se comunican a través de `Espacio` (vía Observer) o de singletons del modelo (`JugadorBueno`).

Parte II: Patrones de Diseño

Introducción a los Patrones de Diseño

En este proyecto se han utilizado tres patrones de diseño para que el código sea más mantenible y extensible: **Factory**, **Strategy** y **Composite**. Cada uno resuelve un problema concreto, y los tres coexisten y se complementan. La mayoría de los patrones ya estaban presentes en concepto desde el Sprint 1, pero se consolidaron y ampliaron al añadir nuevas funcionalidades en los sprints posteriores.

Factory: La Fábrica de Naves

La Idea

Centralizar la creación de naves en un único lugar. Si los parámetros de inicialización cambian (posición, velocidad), solo hay que modificar un fichero. El resto del código trabaja con la abstracción `Naves`, sin conocer las clases concretas.

Evolución del patrón

Sprint 1 — Versión inicial

No existía `NaveFactory`. La nave se creaba directamente en `Espacio.inicializar()`:

```
private void inicializar() {  
    jugador = new Jugador(50, 55);  
    ...  
}
```

Solo había un tipo de nave (`Jugador`), por lo que no había necesidad de una fábrica. Sin embargo, esta lógica estaba acoplada a `Espacio`, y añadir una segunda nave habría requerido modificar el modelo central.

¿Qué motivó el cambio? Al introducir cuatro tipos de nave con geometrías y arsenales distintos, era imprescindible un punto centralizado de creación que desacoplara la elección del tipo (decisión del usuario en `StartFrame`) de la instanciación concreta (responsabilidad de la fábrica).

Versión Final

`NaveFactory` es un Singleton con un único método público `generate(String pNave)`. Todos los parámetros de inicialización (posición inicial `x=50`, `y=55`, `velocidad=1`) están definidos aquí de forma centralizada.

```
public Naves generate(String pNave) {  
    int x = 50, y = 55, velocidad = 1;  
    switch (pNave) {  
        case "Nave1": return new Nave1(x, y, velocidad);  
        case "Nave2": return new Nave2(x, y, velocidad);  
        case "Nave3": return new Nave3(x, y, velocidad);  
    }
```

```

        case "Nave4": return new Nave4(x, y, velocidad);
        default:      return null;
    }
}

```

El cliente (`JugadorBueno`) solo depende de `Naves`. Si se añade una `Nave5`, solo hay que crear la clase y añadir un caso en la fábrica; ningún otro fichero cambia. Esto ya lo demostramos al añadir la `Nave4` en el Sprint 3.

Ventajas del patrón Factory

1. **Encapsulación:** La lógica de creación queda centralizada.
2. **Bajo acoplamiento:** El cliente depende solo de la interfaz `Naves`.
3. **Extensibilidad:** Añadir `Nave5` no afecta al código existente.
4. **Singleton integrado:** Una única instancia de fábrica durante toda la ejecución.
5. **Control global:** Posición y velocidad iniciales en un único lugar.

Strategy: Disparos Intercambiables

La Idea

Cada nave tiene acceso a diferentes tipos de disparo. Unos tienen munición infinita, otros limitada. Unos son un píxel, otros forman figuras compuestas. Strategy permite que cada nave tenga su propio arsenal sin duplicar código y sin condicionales complejos, y además permite cambiar de tipo de disparo en tiempo de ejecución.

Evolución del patrón

Sprint 1 — Versión inicial

No existía el patrón Strategy. El disparo era un objeto `Disparo` simple (un único píxel, munición infinita implícita) creado directamente en `Jugador`. Solo había un tipo de disparo posible y no era intercambiable en tiempo de ejecución.

```

public void disparar() {
    if (!disparo.isActive()) {
        disparo = new Disparo(x, y - 1);
        disparo.setActivo(true);
    }
}

```

Añadir un segundo tipo de disparo habría requerido `if-else` dentro de `Jugador`, acoplando el comportamiento del disparo con la clase de la nave.

¿Qué motivó el cambio? Al diseñar cuatro naves con arsenales diferentes y la posibilidad de cambiar de tipo de disparo en tiempo de ejecución (tecla M), era necesario encapsular cada comportamiento de disparo en una clase propia e intercambiable.

Versión Final

Se define la interfaz `StrategyDisparo` con el contrato común:

```
public interface StrategyDisparo {
    String getTipo();
    int getMunicion();    // -1 = infinita
    boolean gastar();
    boolean tieneMunicion();
    Component crearDisparo(int x, int y);
}
```

Las cuatro estrategias concretas son:

DisparoPixel

Municion infinita. Crea un único `Pixel` en la posición de origen. Disponible en todas las naves como disparo base.

DisparoFlecha

30 disparos. Crea un `Composite` con tres `Pixel` en forma de V invertida: punta en (x, y), alas en (x-1, y+1) y (x+1, y+1). Disponible en Nave1 y Nave3.

DisparoRombo

20 disparos. Crea un `Composite` con cinco `Pixel` en forma de rombo. Disponible en Nave2 y Nave3.

DisparoDoble

25 disparos. Crea un `Composite` con dos `Pixel` en paralelo: (x-1, y) y (x+1, y). Disponible en Nave4.

La clase `Disparo` actúa como **contexto** del patrón: mantiene la lista de estrategias disponibles para la nave actual y el índice de la estrategia activa. Al cambiar de tipo (método `cambiarTipoDisparo()`), recorre la lista en orden circular saltando las que no tienen munición.

El arsenal de cada nave se configura en su constructor:

```
// Nave3 tiene los tres tipos de disparo
ArrayList<StrategyDisparo> estrategias = new ArrayList<>();
estrategias.add(new DisparoPixel());
estrategias.add(new DisparoFlecha());
estrategias.add(new DisparoRombo());
setGestorDisparos(new Disparo(estrategias));
```

Ventajas del patrón Strategy

1. **Intercambiabilidad en tiempo de ejecución:** El jugador cambia entre disparos con la tecla M sin reiniciar nada.
2. **Encapsulación:** Cada estrategia gestiona su propia munición y crea su propia forma.
3. **Eliminación de condicionales:** Sin Strategy habría `if-else` en cada método de

disparo.

4. **Extensibilidad:** Un nuevo disparo (p.ej. `DisparoLaser`) solo requiere implementar la interfaz y añadirlo al constructor de la nave que lo use.
5. **Diferenciación de naves:** El arsenal es parte de la identidad de cada nave, no un añadido externo.

Composite: Píxeles, Naves y Disparos

La Idea

Una nave está hecha de píxeles. Un disparo compuesto también está hecho de píxeles. Composite permite tratarlos de forma uniforme: mover una nave mueve automáticamente todos sus píxeles; comprobar una colisión es simplemente comparar posiciones de píxeles.

Evolución del patrón

Sprint 1 — Versión inicial

No existía el patrón Composite. Cada nave y cada disparo era un único píxel representado por coordenadas `x` e `y`. La detección de colisiones comparaba directamente esas coordenadas con una ventana de tolerancia de 2 píxeles para compensar la discreción del movimiento.

Esto funcionaba perfectamente para el diseño del Sprint 1, pero no era extensible: representar una nave de 10 píxeles con este modelo habría requerido gestionar 10 pares de coordenadas manualmente en cada operación.

¿Qué motivó el cambio? Al diseñar naves con formas geométricas propias (entre 4 y 10 píxeles) y disparos con formas distintas (1, 3 o 5 píxeles), era necesario una estructura que permitiera tratar un conjunto de píxeles como una única entidad manejable.

Versión Final

Se define la interfaz `Component` con el contrato común para todos los elementos del tablero:

```
public interface Component {
    void mover(int dx, int dy, int tipoNave);
    int getRefX();
    int getRefY();
    boolean isActive();
    void setActive(boolean b);
    // + metodos default para notificaciones
}
```

La jerarquía tiene dos implementaciones:

Pixel

La hoja del árbol. Representa una única celda del tablero con posición (`x,y`) y un flag `esProyectil`. Los píxeles de nave solo registran su posición al moverse.

Los píxeles de proyectil notifican a **Espacio** en cada movimiento para que la matriz espejo detecte colisiones, y se desactivan si $y < 0$.

Composite

El nodo interno. Contiene una lista de **Component** (normalmente **Pixel**). Tiene dos modos:

- **Modo nave** ($esProyectil = false$): antes de mover, valida que ningún píxel se saldría del tablero. Si alguno se saldría, no mueve nada. Al mover, desplaza todos los píxeles y notifica el movimiento en bloque a **Espacio**.
- **Modo disparo** ($esProyectil = true$): se mueve libremente; se considera activo mientras al menos un píxel hijo esté activo.

Cada nave define su geometría en el método abstracto **construir()**:

```
// Nave1: forma de T invertida, 10 pixeles
@Override
public void construir() {
    anadirComponente(new Pixel(x,      y,      false, -1));
    anadirComponente(new Pixel(x + 2, y,      false, -1));
    anadirComponente(new Pixel(x,      y + 1, false, -1));
    anadirComponente(new Pixel(x + 1, y + 1, false, -1));
    anadirComponente(new Pixel(x + 2, y + 1, false, -1));
    anadirComponente(new Pixel(x - 1, y + 2, false, -1));
    anadirComponente(new Pixel(x,      y + 2, false, -1));
    anadirComponente(new Pixel(x + 1, y + 2, false, -1));
    anadirComponente(new Pixel(x + 2, y + 2, false, -1));
    anadirComponente(new Pixel(x + 3, y + 2, false, -1));
}
```

Del mismo modo, cada estrategia de disparo construye su forma añadiendo píxeles a un **Composite**:

```
// DisparoFlecha: forma de V invertida, 3 pixeles
Composite comp = new Composite(true, idDisparo);
comp.addComponent(new Pixel(x,      y,      true, idDisparo));
// punta
comp.addComponent(new Pixel(x - 1, y + 1, true, idDisparo));
// ala izq
comp.addComponent(new Pixel(x + 1, y + 1, true, idDisparo));
// ala der
```

La detección de colisiones ya no usa ventana de tolerancia. La **matriz espejo** de **Espacio** registra el id del ocupante de cada celda. Cuando un **Pixel** proyectil se mueve a ($newX$, $newY$), **Espacio** consulta $matriz[newX][newY]$: si hay un enemigo ahí, genera la notificación de colisión (tipos 12, 18, 19) con precisión absoluta de un píxel.

Ventajas del patrón Composite

1. **Uniformidad:** Píxeles individuales y grupos de píxeles se tratan exactamente igual.

2. **Movimiento automático:** Mover un `Composite` mueve todos sus hijos recursivamente.
3. **Colisiones precisas:** Al trabajar a nivel de píxel, no hace falta ventana de tolerancia.
4. **Geometrías flexibles:** Cada nave y cada disparo define su forma añadiendo píxeles; no hay límite de forma ni de tamaño.
5. **Extensibilidad:** Añadir una nueva forma solo requiere definir los píxeles en `construir()` o en `crearDisparo()`.

Cómo Todo Funciona Junto

Los tres patrones se complementan y forman un sistema cohesionado:

- **Factory** decide *qué* nave se crea y con qué parámetros iniciales.
- **Composite** define *cómo* se representa esa nave (y sus disparos) en el tablero.
- **Strategy** determina *qué* tipos de disparo puede usar esa nave y cómo se construye cada uno.

El flujo completo de una partida ilustra cómo interactúan: el usuario elige nave en `StartFrame` y pulsa SPACE. El Controller llama a `JugadorBueno.inicializar("Nave3")`. `JugadorBueno` llama a `NaveFactory.generate("Nave3")` (**Factory**), que devuelve una `Nave3` con su `Composite` de 4 píxeles (**Composite**) y sus tres estrategias de disparo (**Strategy**). Al disparar, la estrategia activa crea un `Composite` de disparo con los píxeles correspondientes; cada píxel, al moverse, notifica a `Espacio`; `Espacio` consulta la matriz espejo y, si hay colisión, notifica a todos los observers. `MainFrame` actualiza la vista; `FlotaEnemigos` elimina al enemigo; `JugadorBueno` elimina el proyectil.

Todo esto ocurre de forma transparente, con cada clase cumpliendo exactamente su responsabilidad.

Conclusiones Finales

La evolución de este proyecto entre el Sprint 1 y la versión final ilustra de forma práctica cómo los patrones de diseño emergen como respuesta a necesidades reales de extensibilidad:

- El Sprint 1 era funcional pero monolítico: `Espacio` concentraba demasiadas responsabilidades y el modelo no escalaba a múltiples naves ni disparos compuestos.
- El Sprint 2 introdujo `JugadorBueno` y `FlotaEnemigos` como clases, y los tres patrones fundamentales del proyecto: Factory (creación), Strategy (comportamiento) y Composite (estructura).
- La versión final distribuye todas las responsabilidades entre clases con roles bien definidos, conectadas mediante el patron Observer (sincronización).

- **MVC + Observer** garantiza que el modelo no conoce las vistas y que las vistas no contienen lógica de negocio. Además, el Observer permite la comunicación entre clases internas del modelo sin necesidad de llamadas innecesarias.
- **Factory** centraliza la creación de naves, facilitando la adición de nuevos tipos sin modificar el resto del sistema.
- **Strategy** encapsula los comportamientos de disparo, permitiendo que sean intercambiables en tiempo de ejecución.
- **Composite** unifica la representación de naves y disparos a nivel de píxel, habilitando colisiones precisas y geometrías arbitrarias.

El resultado es una arquitectura que no solo funciona, sino que está preparada para crecer: añadir una nueva nave, un nuevo tipo de disparo o una nueva pantalla son cambios localizados que no afectan al resto del sistema.